Níveis de Instrumentação e seu Impacto na Observabilidade: Uma Revisão da Literatura Científica

Fagner Silva
Centro de Informática, UFPE
Recife, Brasil
ffcs@cin.ufpe.br

Jackson Raniel
Universidade de Pernambuco - Campus Caruaru
Caruaru, Brasil
jackson.florencio@upe.br

Vinícius Cardoso Garcia
Centro de Informática, UFPE
Recife, Brasil
vcg@cin.ufpe.br

Resumo

Sistemas distribuídos modernos, aplicações nativas da nuvem e ambientes de computação de alto desempenho exigem observabilidade abrangente para garantir a confiabilidade, diagnosticar anomalias de desempenho e otimizar a utilização de recursos. A observabilidade depende fundamentalmente da instrumentação em múltiplas camadas do sistema, onde cada camada introduz compromissos (*trade-offs*) críticos entre profundidade de visibilidade, sobrecarga de desempenho e capacidade diagnóstica. Esta revisão de literatura examina sistematicamente o panorama dos níveis de instrumentação e seus efeitos na observabilidade e no uso de recursos. Analisamos 19 artigos revisados por pares para responder a três questões de pesquisa: mapeamento dos níveis de instrumentação, quantificação do impacto no desempenho e identificação de melhores práticas para minimizar a sobrecarga. Nossos achados revelam uma transição paradigmática de perfis estáticos para abordagens dinâmicas, adaptativas e conscientes de custos. Estruturas dinâmicas como o DTrace atingem sobrecarga próxima de zero quando desativadas, soluções em hardware eliminam artefatos de software na análise de ameaças e estratégias multinível direcionam dinamicamente os recursos de monitoramento durante anomalias. Por fim, esta síntese fornece diretrizes de design e aponta direções futuras, como monitoramento adaptativo guiado por aprendizado de máquina e instrumentação consciente do consumo energético.

Palavras-chave

Observabilidade, Níveis de Instrumentação, Sobrecarga de Desempenho, Revisão Rápida

ACM Reference Format:
Fagner Silva, Jackson Raniel, and Vinícius Cardoso Garcia. 2026. Níveis de Instrumentação e seu Impacto na Observabilidade: Uma Revisão da Literatura Científica. In *Proceedings of 40th Brazilian Symposium on Software Engineering (SBES 2026).* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introdução

Sistemas distribuídos modernos, aplicações nativas da nuvem e ambientes de computação de alto desempenho exigem observabilidade abrangente para garantir a confiabilidade, diagnosticar problemas de desempenho e otimizar a utilização de recursos. A observabilidade — a capacidade de inferir os estados internos de um sistema a partir de suas saídas e emissões de telemetria externas — depende fundamentalmente da instrumentação: a inserção de pontos de medição e sondas (*probes*) que capturam dados de execução, consumo de recursos e padrões comportamentais. No entanto, a instrumentação não é um conceito monolítico; ela pode ser aplicada em múltiplas camadas da pilha computacional, cada uma com características distintas em relação à profundidade de visibilidade, sobrecarga de desempenho (*overhead*) e capacidade diagnóstica.

A escolha do nível de instrumentação representa uma decisão arquitetural crítica com implicações de longo alcance. A instrumentação de grão fino em nível de aplicação pode expor informações semânticas detalhadas sobre a lógica de negócios e estados de variáveis, enquanto o rastreamento em nível de kernel revela a utilização de recursos em todo o sistema operacional e o comportamento do escalonamento de processos. Abordagens baseadas em hardware podem eliminar completamente os artefatos de software, e *frameworks* adaptativos ajustam dinamicamente a intensidade do monitoramento com base nas condições operacionais do sistema. Cada abordagem impõe compromissos (*trade-offs*) inerentes entre a profundidade da observabilidade e a perturbação induzida no desempenho.

Embora levantamentos recentes na literatura abordem aspectos pontuais da telemetria moderna — concentrando-se predominantemente no rastreamento distribuído exclusivo para microsserviços em nuvem ou na aplicação especializada do eBPF em sistemas operacionais —, nota-se uma carência de estudos secundários que tracem um panorama comparativo e vertical abrangente. Persiste uma lacuna de investigações que cruzem sistematicamente as diferentes camadas de abstração do sistema (desde o silício até o código de aplicação), confrontando a riqueza diagnóstica obtida com a quantificação empírica de seus impactos computacionais (latência, utilização de CPU, memória e consumo energético).

Para preencher essa lacuna, esta revisão de literatura examina sistematicamente o panorama dos níveis de instrumentação e seus efeitos na observabilidade e no desempenho do sistema. Analisamos 19 artigos primários revisados por pares para responder a três questões centrais de pesquisa:

- **RQ01:** Quais níveis de instrumentação existem e quais características de observabilidade cada um fornece?
- **RQ02:** Como diferentes abordagens de instrumentação impactam o desempenho do sistema e a utilização de recursos?
- **RQ03:** Quais estratégias e melhores práticas permitem uma observabilidade eficaz minimizando a sobrecarga de desempenho?

Ao sintetizar evidências empíricas e modelos conceituais reportados na literatura recente, esta revisão fornece um arcabouço estruturado para orientar engenheiros e pesquisadores na seleção consciente de técnicas de monitoramento. O restante deste artigo está estruturado da seguinte forma: a Seção 2 discute a fundamentação teórica; a Seção 3 apresenta a metodologia e o protocolo da

revisão rápida; a Seção 4 detalha a taxonomia dos níveis de instrumentação identificados (RQ01); a Seção 5 analisa os impactos na observabilidade, overhead e técnicas adaptativas (RQ02 e RQ03); a Seção 6 sintetiza as discussões e direções de pesquisas futuras; e a Seção 7 conclui o trabalho.

2 Fundamentação Teórica

A instrumentação computacional constitui o alicerce para a medição de desempenho, depuração e compreensão do comportamento dinâmico de sistemas [24]. Contudo, a inserção de código de monitoramento é historicamente limitada pelo chamado "efeito de sonda" (*probe effect*) ou princípio da incerteza da medição de software [7, 20]. Esse fenômeno estabelece que o ato de observar um sistema inevitavelmente altera seu comportamento temporal e espacial: a execução das sondas consome ciclos de processamento, polui as linhas de memória cache, introduz contenção em primitivas de sincronização e pode mascarar ou induzir anomalias artificiais (como condições de corrida e variações de latência) que não ocorreriam na execução não monitorada [20].

A gravidade do efeito de sonda está intimamente associada à chamada "lacuna semântica" (*semantic gap*), formalizada no trabalho pioneiro de Shende et al. [19]. Os autores observaram que as camadas sucessivas de abstração — tais como hardware, sistemas operacionais, máquinas virtuais e interpretadores — criam uma distância conceitual entre a execução física das instruções e a lógica de negócios da aplicação. Essa lacuna impõe um dilema fundamental no projeto de observabilidade: camadas inferiores (hardware e kernel) fornecem medições microscópicas e com mínima perturbação temporal, mas são semanticamente cegas em relação ao fluxo transacional da aplicação; por outro lado, camadas superiores (código-fonte e bytecode) capturam a semântica completa das operações ao custo de maior intrusão e expressivo *overhead* computacional [6, 19].

A evolução histórica da instrumentação reflete os esforços da Engenharia de Software para mitigar essa lacuna semântica sem inviabilizar o desempenho em produção. As abordagens tradicionais das décadas de 1970 a 1990 focavam em instrumentação estática em tempo de compilação para sistemas procedurais e paralelos homogêneos [20]. No entanto, a consolidação de arquiteturas distribuídas, computação em nuvem e microsserviços introduziu novos desafios que tornaram a instrumentação estática proibitiva: a impossibilidade de paradas programadas (*zero downtime*), a heterogeneidade das pilhas tecnológicas e a volatilidade das cargas de trabalho. Esses fatores impulsionaram o desenvolvimento de mecanismos de instrumentação dinâmica e interceptação baseada em *middleware* [18], permitindo a injeção e remoção de sondas sob demanda sem recompilação ou interrupção dos serviços.

Com base nesse corpo teórico, os compromissos de observabilidade podem ser estruturados em quatro dimensões fundamentais, governadas pelo **Princípio da Ortogonalidade** aplicado à taxonomia e ao projeto de sistemas computacionais [16]. Em Engenharia de Software e arquitetura de sistemas, a ortogonalidade postula que as dimensões de um modelo devem ser conceituadas de modo independente e não redundante, garantindo que uma escolha arquitetural em um determinado eixo possa ser realizada sem impor restrições ou acoplamentos espúrios sobre os demais eixos. Nesse sentido, a estruturação multidimensional da observabilidade desdobra-se em quatro eixos ortogonais:

(1) **Camada do Sistema (Eixo Espacial):** a posição na pilha arquitetural onde as medições ocorrem (hardware, kernel, runtime de linguagem ou código de aplicação);
(2) **Características Temporais (Eixo de Vinculação):** o momento de inserção e ativação das sondas, variando de vinculação estática em tempo de compilação a instrumentação dinâmica em tempo de execução sem reinicialização;
(3) **Granularidade dos Dados (Eixo de Resolução):** a densidade da telemetria produzida, oscilando de contadores agregados de baixo custo (*coarse-grained*) até rastreamentos evento a evento com retenção de contexto de chamada (*fine-grained*);
(4) **Política de Ativação (Eixo Operacional):** o regime de funcionamento das medições, alternando entre monitoramento ininterrupto (*always-on*) e amostragem adaptativa ou sob demanda orientada por desvios de desempenho (*cost-aware*).

A adoção explícita do Princípio da Ortogonalidade [16] assegura que o espaço de soluções de observabilidade seja formulado como um produto cartesiano desses eixos. Isso esclarece por que avanços tecnológicos recentes — como o eBPF no kernel ou os agentes dinâmicos de bytecode — combinam pontos distintos dessas dimensões (e.g., sondas dinâmicas de grão fino no kernel ativadas sob demanda), desacoplando a localização física da sonda de sua política de amostragem.

3 Metodologia

As Revisões Rápidas (*Rapid Reviews* - RRs) são estudos secundários projetados para fornecer evidências científicas de forma tempestiva, apoiando a tomada de decisão em ambientes práticos e dinâmicos. Diferente das Revisões Sistemáticas da Literatura (RSL) tradicionais, as RRs otimizam o processo de busca e extração de dados considerando restrições de tempo e esforço, sem comprometer o rigor metodológico.

Esta abordagem é particularmente adequada para este estudo, pois permite sintetizar rapidamente as evidências emergentes sobre os níveis de instrumentação e seus impactos na observabilidade e no desempenho computacional, preenchendo uma lacuna essencial entre a pesquisa acadêmica e a engenharia de sistemas aplicada. O processo desta RR foi conduzido em três fases principais: planejamento, execução e extração, detalhadas nas subseções a seguir.

3.1 Fase de Planejamento

Durante a fase de planejamento, foi estabelecido um protocolo estruturado baseado nas diretrizes de Cartaxo et al. [5]. Este protocolo definiu o problema de pesquisa, a estratégia de busca, os critérios de seleção e o procedimento de extração de dados. Para garantir a transparência e a reprodutibilidade, a ferramenta online *Parsifal* foi utilizada para gerenciar todas as etapas do protocolo.

O objetivo deste estudo é mapear o panorama atual das abordagens de instrumentação, analisando suas capacidades diagnósticas e os compromissos (*trade-offs*) em relação ao consumo de recursos. Para guiar a revisão, foram formuladas as seguintes Questões de Pesquisa (RQs):

- **RQ01:** Quais níveis de instrumentação existem e quais características de observabilidade cada um fornece?
- **RQ02:** Como diferentes abordagens de instrumentação impactam o desempenho do sistema e a utilização de recursos?
- **RQ03:** Quais estratégias e melhores práticas permitem uma observabilidade eficaz minimizando a sobrecarga de desempenho?

Para assegurar a reprodutibilidade da seleção dos estudos e responder rigorosamente às RQs, foram estabelecidos critérios formais de inclusão (*Inclusion Criteria* - IC) e exclusão (*Exclusion Criteria* - EC).

Foram adotados seis critérios de inclusão (IC): **IC01** - Foco primário em observabilidade e técnicas de instrumentação para sistemas de software; **IC02** - Contexto relacionado a camadas da pilha computacional (hardware, kernel, runtime ou aplicação); **IC03** - Avaliação empírica ou quantificação de sobrecarga (*overhead* de CPU, latência, memória ou vazão); **IC04** - Proposta ou análise de ferramentas, *frameworks* ou arquiteturas de telemetria; **IC05** - Artigos publicados no recorte temporal de 2003 a 2025; **IC06** - Artigos redigidos nos idiomas inglês ou português.

Similarmente, foram adotados cinco critérios de exclusão (EC): **EC01** - Artigos duplicados entre as bases científicas; **EC02** - Literatura cinzenta informal e comercial (e.g., postagens em blogs corporativos, fóruns técnicos, manuais comerciais de fornecedores e relatórios de marketing industrial); **EC03** - Foco exclusivo em monitoramento operacional genérico sem abordar conceitos ou *trade-offs* de instrumentação e observabilidade; **EC04** - Estudos onde a instrumentação é apenas mencionada tangencialmente ou utilizada como ferramenta utilitária secundária, não constituindo a contribuição principal; **EC05** - Trabalhos curtos, resumos expandidos e pôsteres com menos de 4 páginas sem detalhamento metodológico ou validação experimental consolidada.

Ressalta-se que, no contexto de Revisões Rápidas orientadas ao suporte prático em Engenharia de Software [5], publicações acadêmicas com metodologia e dados empíricos completos recuperadas via busca em grafos de citação e *snowballing* — a exemplo de teses doutorais institucionais e preprints científicos formais com modelos quantitativos consolidados, como Haghshenas et al. [10] — foram elegíveis por cumprirem integralmente os critérios de rigor (IC01 e IC03), distinguindo-se categoricamente do material comercial opinativo descartado por EC02.

3.2 Fase de Execução

A execução envolveu a busca sistemática na literatura utilizando quatro bases de dados científicas de relevância primária na área de Ciência da Computação e Engenharia de Software: IEEE Xplore, ACM Digital Library, SpringerLink e ScienceDirect. O recorte temporal foi estabelecido entre **2003** e **2025** para capturar as abordagens mais contemporâneas.

A *string* de busca foi construída a partir dos conceitos centrais das RQs e adaptada para a sintaxe específica de cada base, aplicando o filtro de texto completo (*full-text*):

> **String de Busca:**
> `("observability" OR "monitoring") AND ("instrumentation" OR "profiling" OR "tracing") AND ("performance impact" OR "overhead" OR "resource utilization")`

O fluxo de seleção e filtragem dos estudos seguiu as diretrizes do protocolo PRISMA (*Preferred Reporting Items for Systematic Reviews and Meta-Analyses*), conforme ilustrado na Figura 1.

A busca inicial nas quatro bases de dados retornou um total de $N = 348$ artigos (IEEE Xplore: 124, ACM Digital Library: 102, SpringerLink: 76, ScienceDirect: 46). Na etapa de pré-processamento, foram identificados e removidos $N = 62$ estudos duplicados entre as bases, resultando em $N = 286$ artigos únicos para triagem.

O processo de filtragem seguiu três etapas rigorosas:

(1) **Triagem Inicial:** Avaliação de títulos e resumos por um único pesquisador frente aos critérios IC/EC, resultando na exclusão de $N = 224$ trabalhos irrelevantes que não abordavam níveis de instrumentação de sistemas.
(2) **Revisão de Texto Completo:** Os $N = 62$ estudos retidos foram lidos na íntegra. Desses, $N = 48$ foram excluídos por motivos específicos: trabalhos curtos ou literatura cinzenta informal (EC02/EC05, $n = 18$), instrumentação tratada apenas como recurso secundário sem análise direta de suas propriedades (EC04, $n = 21$) e ausência de dados empíricos de *overhead* ou recursos (falha no IC03, $n = 9$).
(3) **Recuperação Adicional e Resolução de Conflitos:** Para expandir a cobertura e mitigar limitações da *string* principal, aplicou-se *Snowballing* (*forward* e *backward*) e busca gráfica pelo *Connected Papers*, adicionando $N = 5$ estudos elegíveis. Em casos de incerteza quanto à inclusão, um segundo pesquisador foi consultado para a decisão final de consenso, resultando no corpus final de $N = 19$ artigos primários.

3.3 Fase de Extração e Relatório

Para cada estudo selecionado na etapa anterior, os dados foram extraídos sistematicamente por um pesquisador e validados de forma cruzada por um segundo autor utilizando os formulários estruturados do *Parsifal*. O mapeamento dos dados extraídos seguiu o alinhamento direto com as Questões de Pesquisa: definições e capacidades operacionais das camadas (para a **RQ01**); quantificações de latência, CPU e memória (para a **RQ02**); e propostas de arquiteturas adaptativas ou *frameworks* conscientes de custos (para a **RQ03**). A fase de relatório consistiu na síntese qualitativa e comparativa dessas informações para compor as discussões estabelecidas nas seções seguintes.

3.4 Ameaças à Validade

A condução desta Revisão Rápida envolveu escolhas metodológicas orientadas pelo equilíbrio entre a tempestividade da entrega e o rigor científico necessário para apoiar decisões práticas [5]. Para assegurar a credibilidade e a transparência dos achados, as potenciais ameaças à validade foram identificadas, estruturadas e tratadas com base no esquema taxonômico de Wohlin et al. [27]:

- **Validade de Construto:** Trata da adequação entre os conceitos teóricos investigados e as medidas operacionais aplicadas [27]. O viés de delimitação do escopo da busca foi mitigado formulando uma *string* com múltiplos sinônimos cobrindo observabilidade, monitoramento, *tracing*, *profiling* e impacto computacional. Além disso, a aplicação de *Snowballing* e a descoberta baseada em grafos de citação

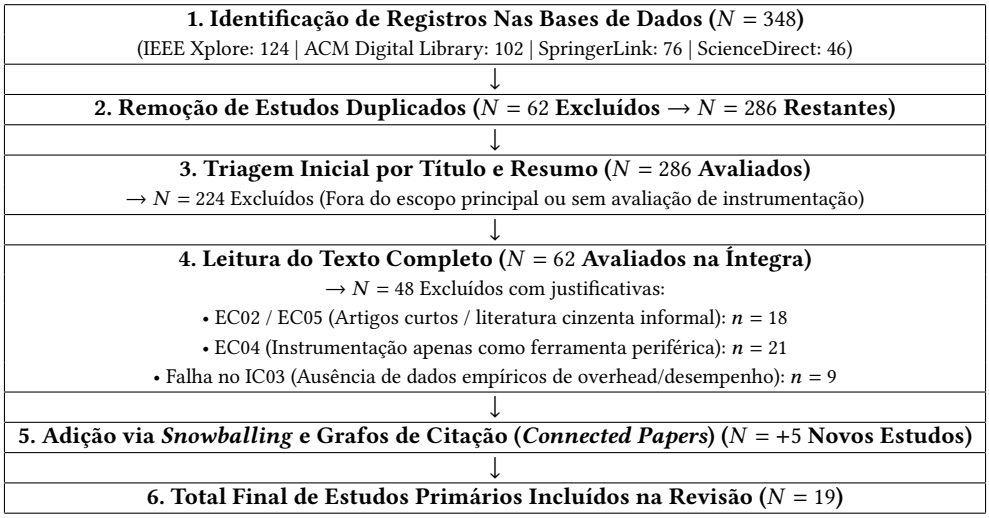


Figure 1: Fluxo do Processo de Seleção e Filtragem de Estudos (Adaptado do Protocolo PRISMA).

(*Connected Papers*) permitiram capturar estudos relevantes que utilizavam terminologias alternativas.

- **Validade Interna:** Refere-se a fatores e vieses não controlados que podem ter afetado a seleção e a extração dos dados [27]. A condução da triagem inicial por apenas um pesquisador representa uma limitação intrínseca ao formato de Revisão Rápida [5]. Para mitigar esse viés, a fase de avaliação do texto completo e a resolução de casos ambíguos contaram com a revisão pareada e validação cruzada por um segundo autor. O uso da ferramenta *Parsifal* assegurou a rastreabilidade e auditabilidade do protocolo.
- **Validade de Conclusão:** Refere-se à capacidade de extrair conclusões corretas a partir dos dados coletados [27]. A acentuada heterogeneidade nas métricas de *overhead* reportadas pelos artigos primários (variando entre incremento de latência, uso de CPU e degradação de vazão) inviabilizou a realização de uma síntese quantitativa ou meta-análise. Essa ameaça foi contornada adotando-se uma síntese qualitativa e uma categorização taxonômica baseada nas camadas de instrumentação e no comportamento temporal.
- **Validade Externa:** Diz respeito à amplitude e capacidade de generalização dos resultados obtidos [27]. A limitação ao recorte temporal (2003–2025) e a seleção de quatro bases acadêmicas indexadas garantem representatividade na literatura científica de Engenharia de Software, embora possam ter desconsiderado relatos práticos presentes na literatura cinzenta (*gray literature*) ou documentações comerciais de ferramentas recentes.

4 Níveis de Instrumentação

A observabilidade em sistemas computacionais modernos é viabilizada pela inserção de pontos de medição em diferentes camadas da pilha de execução [4, 19]. Cada camada oferece uma perspectiva distinta sobre o comportamento do sistema, estabelecendo um compromisso (*trade-off*) inerente entre a riqueza semântica dos dados capturados, a profundidade de visibilidade e a sobrecarga (*overhead*) de desempenho induzida [10, 23]. Enquanto a instrumentação de grão grosso em níveis mais baixos favorece a cobertura global com baixo impacto, a visibilidade fina em níveis superiores expõe detalhes da lógica de negócios ao custo de maior perturbação [8].

Sob a ótica do método taxonômico de Nickerson et al. [16], os quatro níveis de abstração identificados na literatura analisada ($N = 19$) constituem a **dimensão espacial primária** da observabilidade. Essa hierarquia organiza-se desde os recursos físicos de silício até a semântica da aplicação de usuário, conforme esquematizado na Figura 2. As demais características operacionais (como vinculação dinâmica e amostragem adaptativa) operam como eixos ortogonais que qualificam o comportamento de cada ferramenta dentro de sua respectiva camada.

4.1 Instrumentação de Hardware e Host Físico

A instrumentação de nível de hardware representa a camada mais baixa da hierarquia, operando abaixo da pilha de software para monitorar recursos físicos do sistema. Esta abordagem oferece vantagens únicas em termos de observação livre de artefatos e acesso abrangente ao estado.

O sistema LO-PHI (Low-Observable Physical Host Instrumentation) exemplifica a instrumentação baseada em hardware para análise de malware, fornecendo introspecção de máquina física tanto de memória não volátil (disco rígido) quanto volátil (memória do sistema) [21]. O LO-PHI expõe zero artefatos baseados em software e artefatos de hardware mínimos, mantendo capacidades de análise comparáveis às soluções baseadas em software existentes. Essa abordagem analisa significativamente mais binários do que as contrapartes de software porque não pode ser frustrada por técnicas populares de anti-análise. A medição de energia é outro domínio essencial; o trabalho de Ilsche em sistemas de computação de alto desempenho (HPC) quantificou a sobrecarga de medição de energia



Figure 2: Hierarquia das camadas de instrumentação e seus compromissos (*trade-offs*) entre riqueza semântica e sobrecarga computacional.

e o impacto nos recursos, fornecendo insights sobre a utilização de cargas de trabalho analisadas [11].

4.2 Nível de Kernel e Sistema Operacional

A instrumentação de nível de kernel fornece visibilidade sobre chamadas de sistema, decisões de escalonamento, operações de E/S e alocação de recursos.

O DTrace, introduzido por [4] permite a instrumentação dinâmica de softwares de nível de usuário e kernel de forma unificada e segura. Quando não habilitado, o DTrace exibe "efeito de sondagem zero" — o sistema opera como se o DTrace não estivesse presente. O framework suporta dezenas de milhares de pontos de instrumentação, com sistemas pequenos oferecendo cerca de 30.000 pontos apenas no kernel. Tecnologias modernas como eBPF (extended Berkeley Packet Filter) continuam essa evolução. [22] quantifica as diferenças de sobrecarga entre eBPF e SystemTap, orientando a seleção de ferramentas para produção.

4.3 Nível de Máquina Virtual e Runtime

A instrumentação de nível de VM e runtime visa a camada de execução da linguagem, capturando eventos específicos como coleta de lixo (garbage collection) e compilação JIT.

A pesquisa de [8] sobre arquiteturas de observabilidade em camadas para sistemas baseados em JVM descreve a progressão da instrumentação de nível de VM para telemetria em escala de produção. O trabalho analisa o impacto no tempo de resposta e no uso de CPU/memória, fornecendo evidências empíricas dos compromissos inerentes. A instrumentação de runtime permite rastrear padrões de alocação de objetos e eventos de sincronização de threads que são opacos ao nível do kernel.

4.4 Nível de Aplicação e Bytecode

Este nível insere pontos de medição diretamente no código-fonte ou no bytecode, oferecendo a maior riqueza semântica, mas geralmente com a maior sobrecarga.

[23] discute que aplicações complexas mostram comportamentos diferentes em cada parte do código, tornando desejável a instrumentação para separar essas seções e identificar "pontos quentes" (hot spots) de consumo de recursos. [25] descrevem a melhoria da observabilidade de serviços através do monitoramento de variáveis em nível de bytecode, suportando instrumentação estática e dinâmica. O sistema SPADE [2] coleta dados de monitoramento de todos os níveis, incluindo a aplicação, e os armazena em um banco de dados relacional, permitindo correlacionar dados com solicitações semânticas da aplicação. [29] estenderam a programabilidade e observabilidade do eBPF para GPUs, fornecendo múltiplas granularidades de instrumentação ligadas à profundidade de visibilidade e latência.

5 Impacto na Observabilidade

Com relação ao impacto, a profundidade de visibilidade e cobertura sobre a observabilidade, a escolha do nível de instrumentação determina fundamentalmente quais aspectos do comportamento do sistema se tornam observáveis. Diferentes níveis expõem diferentes tipos de eventos, criando uma arquitetura de observabilidade em camadas onde cada camada contribui com insights únicos.

A instrumentação de kernel e de sistema operacional (SO) fornece visibilidade sobre a utilização de recursos em todo o sistema, decisões de escalonamento e padrões de E/S. A capacidade do DTrace de instrumentar tanto o código do kernel quanto o do nível de usuário de maneira unificada permite uma cobertura abrangente do comportamento do sistema [4]. As dezenas de milhares de pontos de instrumentação do framework garantem que virtualmente qualquer evento de kernel ou de espaço de usuário possa ser observado quando necessário [4].

A instrumentação de aplicação e de bytecode expõe sinais semânticos e em nível de variável úteis para depuração e perfilamento (profiling) da lógica de negócios [25]. A arquitetura de observabilidade em camadas de Ghanta [8] demonstra como a instrumentação em nível de VM, de bytecode e de aplicação fornece uma visibilidade progressivamente mais refinada em sistemas baseados em JVM, com cada camada revelando diferentes aspectos do comportamento de execução.

A introspecção de hardware e de host pode revelar artefatos que a instrumentação puramente de software não detecta. A introspecção de máquina física do LO-PHI analisa com sucesso binários que frustram abordagens baseadas em software, demonstrando que a visibilidade em nível de hardware pode expor comportamentos deliberadamente ocultados de observadores em software [21]. Essa capacidade se mostra particularmente importante para análises de segurança e cenários de anti-evasão.

O sistema SPADE ilustra o poder da cobertura multinível [2]. Ao coletar dados de monitoramento de todos os níveis do sistema, incluindo a aplicação, e correlacioná-los com requisições de aplicação semanticamente significativas, o SPADE permite a análise em múltiplos níveis de detalhe simultaneamente [2]. Essa cobertura abrangente permite que os usuários localizem gargalos no sistema e testem hipóteses de desempenho sem a necessidade de múltiplas execuções de experimentos [2].

Com relação às capacidades diagnósticas, os diferentes níveis de instrumentação viabilizam diferentes classes de investigações diagnósticas. A capacidade diagnóstica de uma abordagem de instrumentação depende tanto dos tipos de eventos que ela pode observar quanto da riqueza semântica dos dados capturados.

A instrumentação em nível de kernel se destaca no diagnóstico de problemas de desempenho sistêmicos. Cantrill et al. relatam que o DTrace encontrou sérios problemas de desempenho sistêmico em sistemas de produção que não podiam ser encontrados usando recursos pré-existentes [4]. Os mecanismos de agregação escaláveis e as capacidades de rastreamento especulativo do framework eliminam a necessidade da maior parte do pós-processamento, permitindo o diagnóstico em tempo real. O trabalho posterior de Cantrill descreve um caso em que um multiprocessador simétrico de 64 processadores exibiu misteriosos períodos de atividade do kernel que duravam minutos e faziam com que o desempenho dos benchmarks caísse drasticamente [3]. Melhorias na observabilidade ao nível do kernel mostraram-se essenciais para diagnosticar tais problemas ocultos de desempenho.

A instrumentação em nível de aplicação permite o diagnóstico de problemas de lógica de negócios e problemas de desempenho específicos da aplicação. O rastreamento distribuído do Zipkin melhora significativamente a observabilidade do sistema em arquiteturas de microsserviços, permitindo que os desenvolvedores identifiquem gargalos de desempenho e resolvam problemas de forma mais eficiente [14]. A integração do rastreamento distribuído reduziu o tempo de depuração e aprimorou o monitoramento de desempenho entre os serviços.

A localização da causa raiz representa uma capacidade diagnóstica sofisticada. O sistema MicroRCA de Wu et al. localiza as causas raízes de problemas de desempenho em microsserviços correlacionando sintomas de desempenho da aplicação com a utilização de recursos do sistema, atingindo 89% de precisão e 97% de precisão média ponderada (*mean average precision*) [28]. Notavelmente, o MicroRCA realiza isso sem qualquer instrumentação da aplicação, baseando-se, em vez disso, em métricas de nível de sistema e em um modelo de grafo atribuído de propagação de anomalias.

A instrumentação adaptativa aprimora a capacidade diagnóstica ao direcionar-se dinamicamente a componentes problemáticos. O framework de Mos para monitoramento adaptativo usa modelos de execução para direcionar dinamicamente operações de perfilamento em componentes que exibem degradação de desempenho, garantindo um impacto negligenciável quando a aplicação-alvo funciona conforme o esperado e um impacto mínimo quando determinados componentes apresentam baixo desempenho [15].

5.1 Granularidade e Agregação de Dados

A granularidade dos dados de instrumentação — desde resumos de grão grosso (*coarse-grained*) até rastreamentos de eventos de grão fino (*fine-grained*) — impacta significativamente tanto a eficácia da observabilidade quanto a sobrecarga (*overhead*) do sistema. As estratégias de agregação determinam como os dados brutos de instrumentação são processados e apresentados.

[10] define camadas de instrumentação grossa e fina dentro de um framework consciente dos custos, quantificando seu efeito na profundidade de visibilidade e na latência. Cada camada está vinculada a impactos mensuráveis de desempenho e recursos, fornecendo uma abordagem sistemática para equilibrar os custos de instrumentação com insights de desempenho. Essa abordagem em camadas reconhece que diferentes cenários de diagnóstico exigem diferentes níveis de granularidade.

O mecanismo escalável do DTrace para agregação de dados aborda o desafio de gerenciar o alto volume de saída da instrumentação [4]. Ao realizar a agregação na origem, em vez de exigir um pós-processamento extensivo, o DTrace reduz o *overhead* associado à instrumentação de grão fino. Essa escolha de projeto permite que o framework suporte dezenas de milhares de pontos de instrumentação sem sobrecarregar o sistema com o volume de dados.

O framework *Enhanced Cloud Observability* implementa tanto o log de grão grosso quanto o rastreamento de grão fino por meio

de instrumentação dinâmica [17]. Essa abordagem de dupla granularidade permite que os operadores usem logs leves para monitoramento contínuo, reservando o rastreamento detalhado de grão fino para investigações minuciosas de problemas específicos.

Brandt et al. demonstram que mesmo uma amostragem relativamente grossa pode fornecer insights valiosos. A abordagem de monitoramento de sistema desenvolvida por eles utilizando amostragem de 1Hz não teve impacto significativo nas aplicações, enquanto ainda conseguia resolver características de utilização de recursos [1]. Essa descoberta sugere que uma instrumentação de grão grosso cuidadosamente escolhida pode alcançar uma observabilidade aceitável com o mínimo de overhead.

O sistema EXIST de Wang et al. viabiliza uma observabilidade de rastreamento interna ao serviço extremamente eficiente em datacenters por meio de instrumentação de ultra-baixa sobrecarga [26]. Embora métricas específicas não tenham sido detalhadas nos metadados disponíveis, o trabalho demonstra que a instrumentação otimizada pode alcançar alta observabilidade com custos mínimos de desempenho.

A análise dos mecanismos de coleta de dados e de monitoramento revela que a busca por uma observabilidade abrangente está intrinsecamente atrelada à gestão de restrições de infraestrutura. Esta seção dedica-se a discutir as implicações práticas, os prós e os contras das abordagens mapeadas na literatura, estruturando-se em três eixos fundamentais. Primeiramente, examina-se a divergência operacional entre a instrumentação estática e a dinâmica, avaliando como o momento de inserção das sondas afeta a flexibilidade do sistema. Em seguida, analisa-se a quantificação do *overhead*, consolidando o impacto empírico dessas ferramentas no consumo de recursos. Por fim, discutem-se as estratégias adaptativas e conscientes dos custos, que emergem como soluções sofisticadas para mitigar o impacto concorrencial entre monitoramento e desempenho computacional.

5.2 Instrumentação Estática vs. Dinâmica

As características temporais da instrumentação — se as sondas (probes) são inseridas permanentemente (estática) ou se podem ser ativadas e desativadas em tempo de execução (dinâmica) — afetam fundamentalmente o cálculo de compensação (trade-off) de desempenho.

A instrumentação estática impõe um perfil de overhead constante em sistemas de longa execução. Mos observa que a maioria das soluções de instrumentação exibe comportamento estático ao direcionar-se a um conjunto especificado de componentes e, para aplicações de longa execução, um perfil de sobrecarga constante é indesejável. Tipicamente, a instrumentação estática seria usada apenas durante uma auditoria de desempenho, sacrificando os benefícios de observar constantemente um sistema de produção em favor de um impacto reduzido no desempenho [15].

A instrumentação dinâmica aborda essa limitação permitindo a ativação e desativação de sondas em tempo de execução. O efeito zero de sonda do DTrace quando não ativado representa o caso ideal: o sistema opera exatamente como se a instrumentação não estivesse presente [4]. Essa característica permite ao DTrace manter dezenas de milhares de pontos de instrumentação potenciais sem impor qualquer overhead até que sejam explicitamente ativados. A capacidade de adicionar com segurança muitos pontos de instrumentação sob demanda reduz o impacto em estado estacionário, preservando uma capacidade abrangente de observabilidade.

O framework de instrumentação dinâmica baseado em middleware de Reilly permite a inserção em tempo de execução de serviços de instrumentação sem tirar recursos do ar [18]. Essa capacidade se mostra essencial para aplicações distribuídas onde a instrumentação estática é impraticável devido à distribuição e à necessidade de flexibilidade em tempo de execução. Estudos de caso avaliando o framework avaliaram tanto a eficácia quanto o overhead de desempenho, demonstrando a viabilidade da instrumentação dinâmica em ambientes distribuídos.

A distinção entre instrumentação estática e dinâmica também se aplica ao nível de bytecode. O monitoramento de variáveis em nível de bytecode de Wang et al. suporta instrumentação tanto estática quanto dinâmica durante o uso, fornecendo flexibilidade na forma como a observabilidade é implantada e gerenciada [25].

5.3 Quantificação de Sobrecarga/Overhead

Quantificar o overhead da instrumentação é essencial para tomar decisões informadas entre observabilidade e desempenho. Diferentes abordagens e ferramentas de instrumentação exibem características de sobrecarga variadas.

Volpert et al. fornecem uma comparação exemplar do overhead do eBPF e do SystemTap, quantificando os custos de desempenho associados a esses frameworks de instrumentação dinâmica em nível de kernel [22]. Embora métricas específicas não tenham sido detalhadas nos metadados disponíveis, o trabalho demonstra que a escolha da ferramenta impacta significativamente a sobrecarga e que a medição empírica é necessária para guiar as decisões de implantação em produção.

O framework de monitoramento adaptativo de Mos demonstra um overhead negligenciável quando a aplicação-alvo funciona conforme o esperado, com impacto mínimo quando certos componentes apresentam baixo desempenho [15]. Os resultados experimentais obtidos com o protótipo da ferramenta demonstram a viabilidade dessa abordagem em termos de sobrecarga induzida. Essa estratégia adaptativa transforma efetivamente o perfil de overhead de constante (como na instrumentação estática) para condicional (ativado apenas quando necessário).

Brandt et al. relatam que sua abordagem de monitoramento de sistema usando amostragem de 1Hz não teve impacto significativo nas aplicações [1]. Essa descoberta fornece um ponto de dados concreto: a amostragem a 1Hz — uma taxa suficiente para resolver características de utilização de recursos — impõe um overhead negligenciável.

O NEOS (Non-Intrusive Edge Observability Stack) de Kumar et al. reduz a quantidade de instrumentação de código necessária em até 80% em comparação com a instrumentação manual [13]. Embora essa métrica aborde principalmente o esforço de desenvolvimento em vez do overhead em tempo de execução, a redução no código de instrumentação provavelmente se correlaciona com um menor impacto em tempo de execução.

A instrumentação baseada em hardware do LO-PHI fornece capacidades de análise comparáveis às soluções baseadas em software existentes, enquanto expõe zero artefatos baseados em software

e mínimos artefatos de hardware [21]. Essa abordagem troca a complexidade de implementação por uma menor perturbação em tempo de execução, demonstrando que a instrumentação em nível de hardware pode minimizar o overhead visível pelo software.

5.4 Abordagens Adaptativas e Conscientes dos Custos

Estratégias de instrumentação adaptativas e conscientes dos custos representam abordagens sofisticadas para gerenciar a compensação entre observabilidade e desempenho. Essas técnicas ajustam dinamicamente a intensidade do monitoramento com base no comportamento do sistema ou modelam explicitamente o valor da informação contra o custo da instrumentação.

O framework consciente dos custos de Haghshenas et al. equilibra explicitamente os custos de instrumentação com os insights de desempenho [10]. O framework define camadas de instrumentação grossas e finas, quantificando seu efeito na profundidade de visibilidade e na latência, e vincula cada camada a impactos mensuráveis de desempenho e de recursos. Essa abordagem fornece uma metodologia sistemática para selecionar configurações de instrumentação que atendam aos objetivos de observabilidade dentro das restrições de recursos.

O framework de monitoramento adaptativo de Mos usa modelos de execução para direcionar dinamicamente as operações de perfilamento em componentes que exibem degradação de desempenho [15]. Essa estratégia garante um overhead negligenciável durante a operação normal e aumenta a intensidade do monitoramento apenas quando necessário. A abordagem aborda o perfil de sobrecarga constante indesejável da instrumentação estática, tornando a intensidade do monitoramento responsiva ao comportamento do sistema.

O framework ENHANCED CLOUD OBSERVABILITY implementa instrumentação dinâmica com log de grão grosso e rastreamento de grão fino, conectando as escolhas de instrumentação à profundidade de visibilidade e aos impactos de desempenho [17]. Essa abordagem multinível permite que os operadores mantenham um monitoramento contínuo leve, enquanto reservam a dispendiosa instrumentação de grão fino para investigações direcionadas.

Abordagens adaptativas reconhecem que nem todos os estados do sistema exigem o mesmo nível de observabilidade. Ao concentrar os recursos de instrumentação em componentes ou períodos de tempo problemáticos, essas estratégias alcançam melhores compensações gerais do que as abordagens de instrumentação uniforme. O principal desafio reside no desenvolvimento de heurísticas ou modelos eficazes para determinar quando e onde intensificar o monitoramento.

6 Discussões e Trabalhos Futuros

A Tabela 1 sintetiza as principais características das abordagens de instrumentação analisadas na literatura ($N = 19$), estruturando-as a partir da matriz taxonômica multidimensional. Fundamentada no Princípio da Ortogonalidade [16], a matriz de classificação desacopla a camada física de execução (nível do sistema) de suas propriedades temporais, estratégias de agregação e políticas adaptativas de custo. Essa abordagem permite aos projetistas identificar claramente os compromissos de cada proposta, comparando suas capacidades diagnósticas, a ordem de grandeza do impacto no desempenho e seus domínios primários de aplicação.

Com base na literatura sintetizada, surgem várias melhores práticas para o projeto e a implantação de sistemas de instrumentação que equilibram a observabilidade com o desempenho.

Adequar o nível de instrumentação à necessidade diagnóstica: Instrumente a camada mais grossa (coarse) que ainda forneça o sinal necessário para minimizar o volume de dados e a intervenção [23]. Para problemas de desempenho sistêmicos, a instrumentação em nível de kernel pode ser suficiente [4], enquanto problemas específicos da aplicação podem exigir visibilidade em nível de bytecode ou de aplicação [14, 25].

Preferir a instrumentação dinâmica à estática: A instrumentação dinâmica permite uma operação com overhead zero quando não está em uso, preservando uma capacidade abrangente de observabilidade [4]. Essa abordagem permite que os sistemas mantenham extensos pontos de instrumentação sem impor uma sobrecarga constante [4].

Empregar estratégias de ativação adaptativa: Ative as sondas (probes) apenas ao investigar anomalias ou direcione a amostragem para componentes problemáticos para evitar o overhead constante [10, 15]. O monitoramento adaptativo que ativa o perfilamento apenas para componentes com mau funcionamento pode manter a sobrecarga negligenciável durante a operação normal [15].

Aproveitar a agregação e o pré-processamento: Utilize a agregação no kernel (in-kernel) ou no processo (in-process) para reduzir a telemetria exportada e os custos de pós-processamento [4]. O mecanismo de agregação escalável do DTrace demonstra como o processamento na origem pode gerenciar dados de instrumentação de alto volume [4].

Considerar arquiteturas de instrumentação multinível: Combine a instrumentação em múltiplos níveis para alcançar uma observabilidade abrangente [2, 8]. A abordagem multinível do SPADE com correspondência automática de granularidade mostra como diferentes camadas podem ser integradas de forma eficaz [2].

Aplicar a seleção consciente dos custos: Adote políticas de instrumentação conscientes dos custos que troquem a fidelidade do sinal pelo orçamento de recursos e priorizem as medições de maior valor sob restrições de recursos [10]. Modelar explicitamente a compensação (trade-off) custo-benefício permite uma otimização sistemática [10].

Usar instrumentação de hardware para observação livre de artefatos: Quando os artefatos de software forem problemáticos (e.g., análise de segurança, cenários de anti-evasão), a instrumentação em nível de hardware pode fornecer observabilidade comparável sem perturbações visíveis pelo software [21].

Implementar granularidade em camadas: Mantenha um monitoramento contínuo de grão grosso para a observabilidade de linha de base (baseline), enquanto reserva a instrumentação de grão fino para investigações direcionadas [10, 17]. Essa abordagem em camadas equilibra a visibilidade contínua com as restrições de desempenho [17].

A literatura sugere várias direções promissoras para pesquisas futuras em instrumentação e observabilidade.

Instrumentação adaptativa aprimorada por aprendizado de máquina: Embora as abordagens adaptativas atuais utilizem modelos de execução e heurísticas [15], o aprendizado de máquina poderia permitir

Table 1: Mapeamento Sintético e Comparativo das Abordagens de Instrumentação na Literatura ($N = 19$)

Nível / Camada	Ferramentas / Propostas	Características de Observabilidade	Impacto no Desempenho	Casos de Uso Principais	Ref. e Anos
Hardware & Host Físico	LO-PHI, Medição em HPC	Introspecção de memória volátil e não-volátil; zero artefatos de software; métricas de energia em nível de barramento.	Zero perturbação no software alvo; artefatos mínimos em hardware.	Análise de *malware*, segurança cibernética e auditoria energética em HPC.	[21] (2016) [11] (2020)
Kernel & Sistema Operacional	DTrace, eBPF, SystemTap, BPF Tools	Instrumentação unificada kernel/espaço de usuário; dezenas de milhares de sondas dinâmicas; rastreamento de syscalls e I/O.	Efeito de sonda zero quando inativo (DTrace); baixo *overhead* e agregação na origem.	Diagnóstico sistêmico em produção; análise de latência e contenção de recursos.	[4] (2004) [3] (2006) [9] (2019) [22] (2025)
VM & Runtime	Arquiteturas JVM, Middleware Dinâmico	Rastreamento de alocação de objetos, compilação JIT, eventos de GC e sincronização de *threads*.	*Overhead* variável por camada; exige amostragem ou ativação condicional.	Telemetria em escala de produção; diagnóstico em sistemas Java/componentizados.	[18] (2006) [8] (2017)
Aplicação & Bytecode	SPADE, Zipkin, eGPU, Bytecode Variable Monitoring, App Tuning	Visibilidade em nível de variável, rastreamento distribuído fim-a-fim de requisições, telemetria em GPUs e hotspots de código.	*Overhead* de médio a elevado; exige balanceamento rigoroso de granularidade.	Localização de causa raiz, microsserviços, depuração semântica e GPU *offloading*.	[2] (2003) [23] (2020) [14] (2024) [25] (2025) [29] (2025)
Adaptativo & Consciente de Custos	Cost-Aware Framework, Adaptive Monitoring, MicroRCA, NEOS, EXIST	Ativação sob demanda; amostragem adaptativa baseada em anomalias; correlação de métricas de SO sem alteração de código.	*Overhead* negligenciável em operação normal; custo condicional durante falhas.	Ambientes de borda (*Edge*), *datacenters* e localização automatizada de causa raiz.	[15] (2004) [28] (2020) [10] (2023) [13] (2023) [26] (2025)
Resiliência & AIOps	Cloud Resilience Architecture	Previsão de falhas com suporte a aprendizado de máquina e mitigação proativa de anomalias.	Custos computacionais deslocados para algoritmos preditivos fora do caminho crítico.	Sistemas de alta disponibilidade na nuvem e resiliência preditiva.	[12] (2025)

previsões mais sofisticadas de quando e onde intensificar o monitoramento. Jalukuri observa que a recuperação preditiva baseada em aprendizado de máquina representa um avanço nas práticas de resiliência [12], e técnicas semelhantes poderiam otimizar a ativação da instrumentação.

Correlação e análise entre camadas (cross-layer): Embora exista instrumentação multinível [2, 8], técnicas mais sofisticadas para correlacionar eventos entre camadas poderiam revelar interações complexas. A inferência causal automatizada entre níveis de instrumentação poderia aprimorar as capacidades de análise de causa raiz.

Instrumentação para arquiteturas emergentes: O trabalho de Yang et al. sobre observabilidade de GPUs [29] demonstra a necessidade de estender os conceitos de instrumentação para novos paradigms computacionais. À medida que a computação heterogênea, os sistemas quânticos e as arquiteturas neuromórficas emergem, as técnicas de instrumentação devem evoluir de acordo.

Padronização e interoperabilidade: A pilha NEOS de Kumar et al. aproveita ferramentas de código aberto populares como OpenTelemetry, Grafana, Prometheus, Jaeger e Loki [13]. Esforços contínuos de padronização poderiam melhorar a interoperabilidade e reduzir o esforço de instrumentação em sistemas diversos.

Verificação formal da segurança da instrumentação: Embora o DTrace enfatize a segurança absoluta [4], métodos formais poderiam fornecer garantias mais fortes sobre a correção e a não interferência da instrumentação, particularmente para sistemas de missão crítica ou críticos para a segurança (safety-critical).

Instrumentação consciente do consumo de energia: O trabalho de Ilsche sobre medições de energia [11] destaca a importância de compreender a pegada energética da instrumentação. À medida que a eficiência energética se torna cada vez mais crítica, as estratégias de instrumentação devem considerar explicitamente o consumo de energia juntamente com o overhead de desempenho.

7 Conclusão

Esta revisão de literatura examinou o panorama dos níveis de instrumentação e seu impacto na observabilidade, sintetizando as descobertas de 19 artigos revisados por pares que abrangem introspecção de hardware, rastreamento de kernel, monitoramento em tempo de execução e instrumentação de aplicações. As evidências demonstram que a instrumentação pode ser aplicada em múltiplas camadas do sistema, cada uma oferecendo compensações (trade-offs) distintas entre a profundidade de observabilidade e a sobrecarga de desempenho.

As principais descobertas incluem:

Frameworks de instrumentação dinâmica como o DTrace alcançam overhead zero quando desativados, enquanto fornecem observabilidade abrangente quando ativados [4];

Abordagens baseadas em hardware eliminam artefatos de software ao mesmo tempo em que mantêm a capacidade diagnóstica [21];

Estratégias adaptativas e conscientes dos custos concentram os recursos de monitoramento nos componentes problemáticos,

alcançando uma sobrecarga negligenciável durante a operação normal [10, 15];

Arquiteturas de instrumentação multinível fornecem visibilidade abrangente ao combinar insights de diferentes camadas do sistema [2, 8]; e

A escolha da ferramenta impacta significativamente o overhead, sendo necessária a comparação empírica para guiar a implantação em produção [22].

A literatura revela uma evolução clara da instrumentação estática e sempre ativa (always-on) em direção a abordagens dinâmicas, adaptativas e conscientes dos custos, que gerenciam de forma inteligente o trade-off entre observabilidade e desempenho. As melhores práticas enfatizam a adequação do nível de instrumentação à necessidade diagnóstica, a preferência pela instrumentação dinâmica em relação à estática, o emprego de ativação adaptativa, o aproveitamento da agregação e a aplicação de estratégias de seleção conscientes dos custos.

As direções de pesquisas futuras incluem instrumentação adaptativa aprimorada por aprendizado de máquina, técnicas de correlação entre camadas, instrumentação para arquiteturas emergentes, esforços de padronização, verificação formal da segurança da instrumentação e estratégias de instrumentação conscientes do consumo de energia. À medida que os sistemas continuam a crescer em complexidade e escala, técnicas sofisticadas de instrumentação continuarão essenciais para manter a observabilidade enquanto preservam o desempenho.

A tensão fundamental entre a observabilidade e o overhead de desempenho não pode ser eliminada, mas as técnicas mapeadas nesta revisão demonstram que ela pode ser gerenciada de forma eficaz. Ao selecionar os níveis de instrumentação apropriados, empregar a ativação dinâmica e adotar estratégias adaptativas, as organizações podem alcançar uma observabilidade abrangente com um impacto de desempenho aceitável — variando tipicamente de um overhead quase zero para instrumentação seletiva a custos gerenciáveis para monitoramento de grão fino. O avanço contínuo das técnicas de instrumentação será crítico para garantir a confiabilidade, o desempenho e a segurança dos sistemas de computação do futuro.

Uso de Ferramentas de IA Modelos de Linguagem de Grande Porte (LLMs) e ferramentas assistidas por IA, especialmente Gemini e Grammarly, foram utilizados exclusivamente para edições linguísticas menores e polimento de texto sob a supervisão direta dos autores. As imagens utilizadas neste artigo foram geradas por Gemini AI. A IA não gerou nenhuma parte do design da pesquisa, interpretação ou raciocínio científico. Os autores assumem total responsabilidade pelo conteúdo e conclusões deste manuscrito, e todo o trabalho conceitual e analítico foi conduzido inteiramente por autores humanos.

8 Declarações

Financiamento: Não aplicável.
Conflitos de Interesse: Não aplicável.
Aprovação Ética e Consentimento para Participação: Não aplicável, visto que este estudo não envolve participantes humanos ou animais.
Consentimento para Publicação: Não aplicável.
Disponibilidade de Dados: Não aplicável, uma vez que nenhum dado externo foi gerado nesta fase deste estudo.
Disponibilidade de Materiais: Não aplicável.
Contribuição dos Autores: O Autor 1 conduziu a concepção do estudo, busca, triagem e síntese. O Autor 2 orientou a metodologia e realizou a revisão crítica do texto. O Autor 3 supervisionou o estudo e contribuiu para a consolidação das seções de discussão e conclusão. Todos os autores leram, revisaram e aprovaram a versão final do manuscrito.

Disponibilidade de Artefatos

Para garantir a reprodutibilidade, auditabilidade e transparência desta Revisão Rápida, o pacote de replicação completo contendo o protocolo do Parsifal, a string de busca adaptada por base, a lista dos $N = 348$ registros recuperados, os critérios de inclusão/exclusão aplicados individualmente em cada etapa e a matriz de extração de dados contendo os $N = 19$ estudos primários foi disponibilizado publicamente no repositório no GitHub.

Referências

[1] James M. Brandt, Jeremy Enos, Ann Gentile, and Mike Showerman. 2018. *Application Performance Insights via System Monitoring.* Technical Report SAND2018-6888C. Sandia National Laboratories, Albuquerque, NM, USA. https://www.osti.gov/servlets/purl/1515734

[2] Aaron B. Brown and David A. Patterson. 2003. Analyzing Application Performance Using a System Monitoring Database. In *Proceedings of the International Conference of the Computer Measurement Group (CMG).* University of California at Berkeley, Dallas, TX, USA, 1–10.

[3] Bryan M. Cantrill. 2006. Hidden in Plain Sight: Improvements in the Observability of Software. *ACM Queue* 4, 1 (2006), 26–36. doi:10.1145/1117389.1117401

[4] Bryan M. Cantrill, Michael W. Shapiro, and Adam H. Leventhal. 2004. Dynamic Instrumentation of Production Systems. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC).* USENIX Association, Boston, MA, USA, 15–28. https://www.usenix.org/legacy/event/usenix04/tech/general/cantrill.html

[5] Bruno Cartaxo, Gustavo Pinto, and Sérgio Soares. 2018. The Role of Rapid Reviews in Supporting Decision-Making in Software Engineering Practice. In *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering (EASE).* Association for Computing Machinery, New York, NY, USA, 24–34. doi:10.1145/3210459.3210462

[6] Norbert Debes. 2010. Application Instrumentation and End-to-End Tracing. In *Secrets of the Oracle Database.* Apress, Berkeley, CA, USA, 251–267. doi:10.1007/978-1-4302-1953-8_23

[7] Vicente García, José M. López, Daniel F. García, and Javier Tuya. 2000. Model-Driven Monitoring Support for the Multi-View Performance Analysis of Parallel Embedded Applications. *Performance Evaluation* 40, 1–3 (2000), 197–214. doi:10.1016/S0166-5316(99)00059-0

[8] Sriram Ghanta. 2017. *Layered Observability Architectures for JVM-Based Systems: From VM-Level Instrumentation to Production-Scale Telemetry.* Technical Report. Technical Report.

[9] Brendan Gregg. 2019. *BPF Performance Tools: Linux System and Application Observability.* Addison-Wesley Professional, Boston, MA, USA.

[10] Amir Hossein Haghshenas, Naser Ezzati-Jivan, and Michel Dagenais. 2023. Balancing Instrumentation Costs and Performance Insights: A Cost-Aware Approach to Execution Monitoring. *SSRN Electronic Journal* (2023), 1–14. doi:10.2139/ssrn.4556986

[11] Thomas Ilsche. 2020. *Energy Measurements of High Performance Computing Systems: From Instrumentation to Analysis.* Ph. D. Dissertation. Dresden University of Technology, Dresden, Germany. doi:10.25368/2021.16

[12] Suresh Jalukuri. 2025. The Evolution of Cloud Resilience: Observability, Automation, and High Availability. *Journal of Computer Science and Technology Studies* 7, 1 (2025), 112–124. doi:10.32996/jcsts.2025.7.5.7

[13] Abhijit Kumar, Tauseef Ahmed, Konica Saini, and Jay Kumar. 2023. NEOS: Non-intrusive Edge Observability Stack Based on Zero Trust Security Model for Ubiquitous Computing. In *Proceedings of the 2023 IEEE International Conference on Edge Computing and Communications (EDGE).* IEEE, Chicago, IL, USA, 79–84. doi:10.1109/EDGE60047.2023.00023

[14] Okpako Marvis, Olanriwaju Babatunde, and Eguavoen Victor Osasu. 2024. Exploring Tracing in Microservice Applications: Leveraging Zipkin for Enhanced Observability. *International Journal of Research and Scientific Innovation (IJRSI)* 11, 11 (Nov. 2024), 384–395. doi:10.51244/IJRSI.2024.11110030

[15] Adrian Mos. 2004. *A Framework for Adaptive Monitoring and Performance Management of Component-Based Enterprise Applications.* Ph. D. Dissertation. School of Computing, Dublin City University, Dublin, Ireland.

[16] Robert C. Nickerson, Upkar Varshney, and Jan Muntermann. 2013. A Method for Taxonomy Development and Its Application in Information Systems. *European Journal of Information Systems* 22, 3 (2013), 336–359. doi:10.1057/ejis.2012.26

[17] P. Ramesh. 2025. ENHANCED CLOUD OBSERVABILITY: An Integrated Framework for Incident Analysis Using RED, USE, and MDC Methodologies. *International Journal of Information Technology and Management Information System (IJITMIS)* 16, 1 (Jan. 2025), 51–63. doi:10.34218/ijitmis_16_01_051

[18] Darren Reilly. 2006. *A Dynamic Middleware-Based Instrumentation Framework to Assist the Understanding of Distributed Applications.* Ph. D. Dissertation. Liverpool John Moores University, Liverpool, UK. doi:10.24377/LJMU.T.00005815

[19] Sameer Suresh Shende. 2001. *The Role of Instrumentation and Mapping in Performance Measurement.* Ph. D. Dissertation. Department of Computer and Information Science, University of Oregon, Eugene, OR, USA.

[20] Anand Sivasubramaniam, Aman Singla, Umakishore Ramachandran, and H. Venkateswaran. 1994. *A Comparative Evaluation of Techniques for Studying Parallel System Performance.* Technical Report GIT-CC-94/38. College of Computing, Georgia Institute of Technology, Atlanta, GA, USA.

[21] Chad Spensky, Aravind Prakash, Justin C. Klein, and Robert J. Walls. 2016. LOPHI: Low-Observable Physical Host Instrumentation for Malware Analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS).* Internet Society, San Diego, CA, USA, 1–15. doi:10.14722/ndss.2016.23121

[22] Arthur Volpert, Andreas Rain, and Stefan Wesner. 2025. Towards eBPF Overhead Quantification: An Exemplary Comparison of eBPF and SystemTap. In *Companion of the 2025 ACM/SPEC International Conference on Performance Engineering (ICPE '25).* Association for Computing Machinery, New York, NY, USA, 120–127. doi:10.1145/3680256.3721311

[23] Ondřej Vysocký, Lubomír Říha, Martin Golasowski, Tomáš Brzobohatý, and Václav Švaton. 2020. Overview of Application Instrumentation for Performance Analysis and Tuning. In *Euro-Par 2019: Parallel Processing Workshops (Lecture Notes in Computer Science, Vol. 11997).* Springer, Cham, Switzerland, 161–173. doi:10.1007/978-3-030-43222-5_14

[24] Abdul Waheed. 1997. *Instrumentation System Design, Modeling, and Evaluation.* Ph. D. Dissertation. Department of Computer Science, Michigan State University, East Lansing, MI, USA.

[25] Taizheng Wang, Chunyang Ye, Hui Zhou, Wei Chang, and Chaoyi Li. 2025. Enhancing Service Observability Through Bytecode-Level Variable Monitoring. In *Proceedings of the 2025 IEEE International Conference on Web Services (ICWS).* IEEE, Helsinki, Finland, 880–882. doi:10.1109/icws67624.2025.00114

[26] Xinkai Wang, Xiaofeng Hou, Chao Li, Yuancheng Li, Du Liu, Guoyao Xu, Guodong Yang, Liping Zhang, Yuemin Wu, and Xiaopeng Yuan. 2025. EXIST: Enabling Extremely Efficient Intra-Service Tracing Observability in Datacenters. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*, Vol. 2. Association for Computing Machinery, New York, NY, USA, 355–372. doi:10.1145/3676641.3716283

[27] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering.* Springer Science & Business Media, Berlin, Heidelberg. doi:10.1007/978-3-642-29044-2

[28] Li Wu, Johan Tordsson, Jasmin Bogatinovski, Alexander Acker, and Odej Kao. 2020. MicroRCA: Root Cause Localization of Performance Issues in Microservices. In *Proceedings of the IEEE/IFIP Network Operations and Management Symposium (NOMS).* IEEE, Budapest, Hungary, 1–9. doi:10.1109/NOMS47738.2020.9110353

[29] Yiwei Yang, Tong Yu, Yusheng Zheng, and Andrew Quinn. 2025. eGPU: Extending eBPF Programmability and Observability to GPUs. In *Proceedings of the 4th Workshop on Heterogeneous Composable and Disaggregated Systems (HCDS).* Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3723851.3726984